

ARQUITETURA DE SOFTWARE • DESIGN PATTERNS • SOLID

Sistema de Trocas de Cartas

Refatoração de uma API de troca de cartas aplicando padrões de projeto (design patterns)

NestJS

Prisma

PostgreSQL

Strategy · State · Observer



APRESENTAÇÃO · A EQUIPE

Quem construiu o sistema

Cinco integrantes — cada um responsável por um módulo e pelo padrão de projeto aplicado nele.



Fábio Henrique

Módulo Trades · Observer + camadas



Gabriel Baldoni

Trade Proposal · State



Gabriel Renato

Wishlist · Diagramas



Bruno Nunes

Eventos · Observe · Organização do projeto



Ian Marques

Wishlist · Strategy

Roteiro:

Diagramas



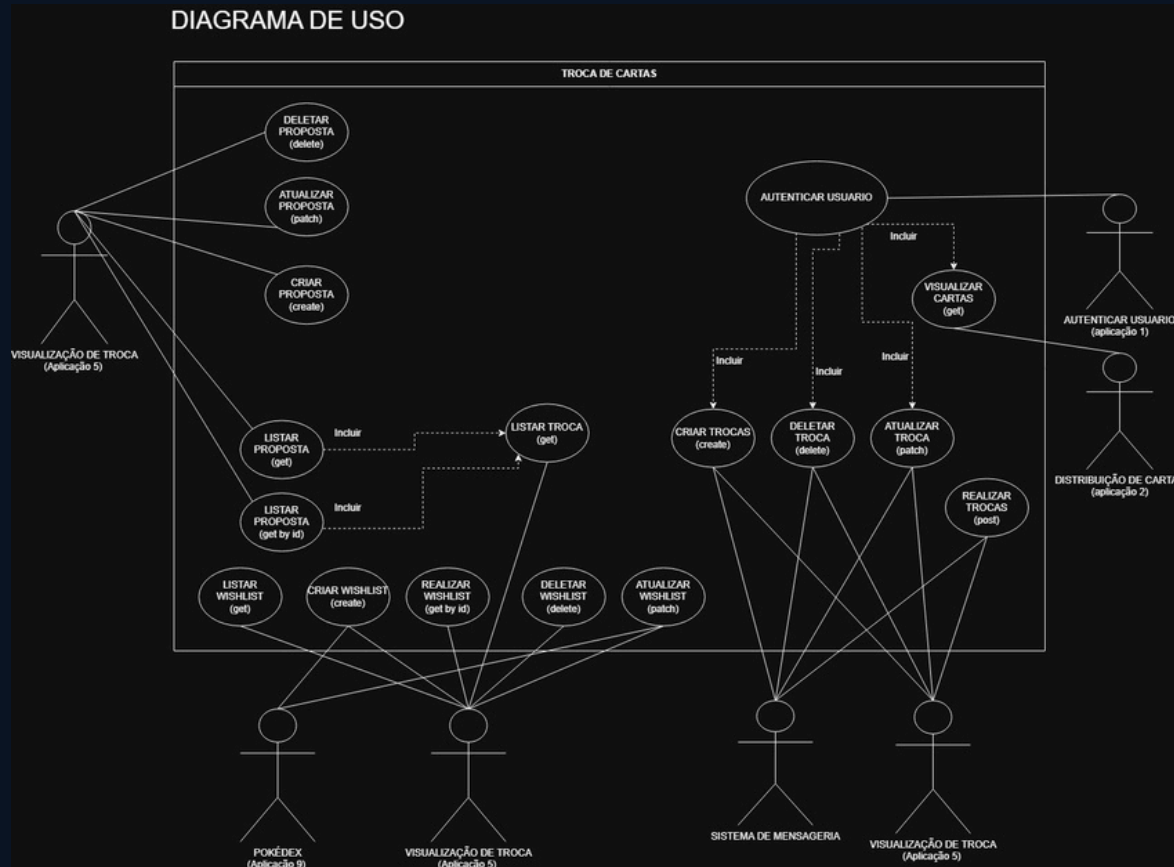
Design Patterns



Código rodando



Diagrama de Casos de Uso

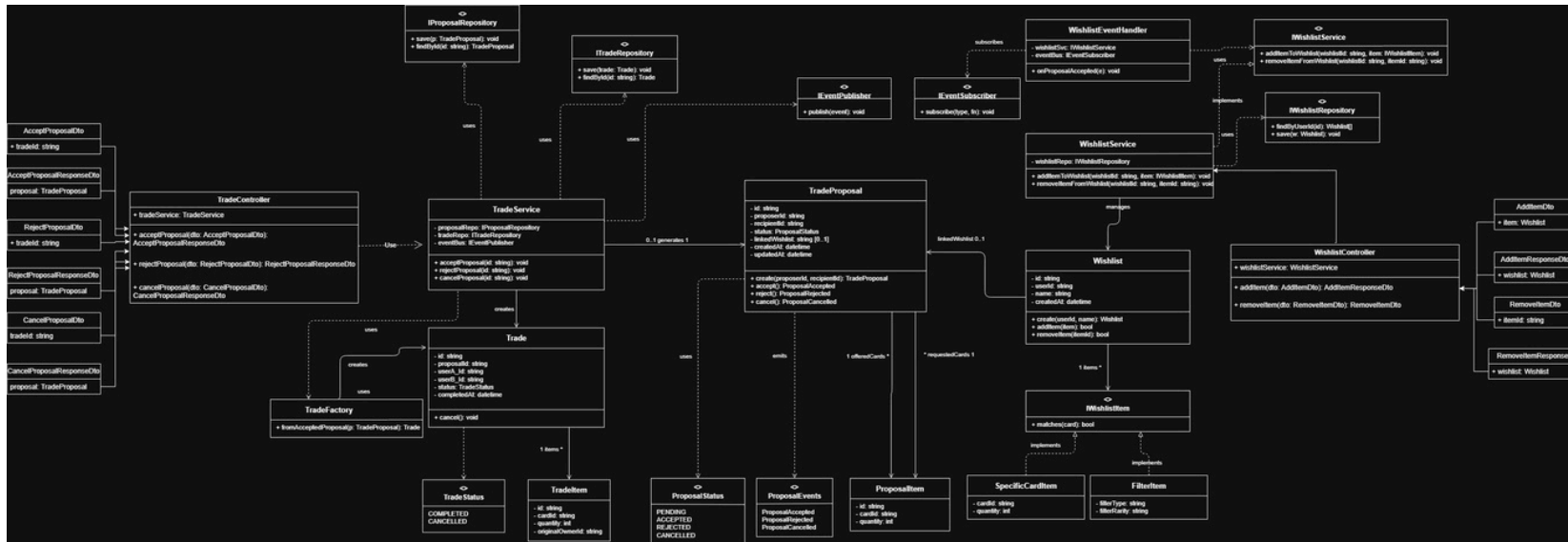


O domínio

- ◆ Tudo gira em torno de **Troca de Cartas**: propostas, trocas e wishlists.
- ◆ **Atores externos** integram o sistema — Pokédex, Mensageria, Autenticação e Distribuição de cartas.
- ◆ Cada caso de uso tem seus endpoints respectivos e funcionalidades



Diagrama de Classes (v5 — refatorado com SOLID)



Onde os padrões entram

- ◆ Serviços conversam por **interfaces** — `IWishlistItem`, `IEventPublisher`, `IEventSubscriber`.
- ◆ **Strategy** na `Wishlist`, **State** na `TradeProposal`, **Observer** no event bus.
- ◆ Repositórios isolam o Prisma — a regra de negócio não conhece o ORM.



3 — O QUE IMPLEMENTAMOS DE NOVO

Design Patterns aplicados

Três frentes de refatoração — cada padrão remove um *if/else* que crescia sem controle e deixa o código aberto para extensão.

COMPORTAMENTAL

Strategy

src/wishlist

Cada tipo de “item desejado” encapsula sua própria regra de matches() — carta específica ou filtro.

Open/Closed

COMPORTAMENTAL

State

src/trade-proposal

O ciclo de vida da proposta vira uma máquina de estados — cada status decide quais transições são válidas.

Single Responsibility

ESTRUTURAIS + GOF

Camadas + Observer

src/trades · providers/events

Repository, Service, DTO, DI, Module e um event bus baseado em interfaces desacoplando os módulos.

Dependency Inversion



Item desejado sem ifs gigantes

O problema

DOIS TIPOS BEM DIFERENTES

Carta específica — “quero o cardId = abc-123”.

Filtro — “quero qualquer Pokémon raridade Rare Holo”.

- 1 **IWishlistItem** define só `matches(cardId): boolean`.
- 2 **SpecificCardItem** / **FilterItem** implementam a própria regra.
- 3 **WishlistItemFactory** é o único ponto que conhece os tipos concretos.

O contrato comum

```
export interface IWishlistItem {  
  matches(cardId: string): boolean;  
}  
  
class SpecificCardItem implements IWishlistItem {  
  matches(cardId) {  
    return this.cardId === cardId;           /= regra isolada  
  }  
}
```

Adicionar um novo tipo de item = nova classe + um case na factory. Service, controller e event handler não mudam.



Ciclo de vida da proposta sem if manual

O problema

VALIDAÇÃO DUPLICADA NO SERVICE

O mesmo `if (status !== PENDING) throw...` se repetia em `update()` e `acceptProposal()`. Cada novo status forçava mexer no service.

- P** **Pending** — único transitável: aceita, rejeita, cancela.
- A** **Accepted / Rejected / Cancelled** — terminais: lançam `ConflictException`.
- **TradeProposalEntity** delega para o estado via `ProposalStateFactory`.

Estado decide a transição

/= antes — repetido no service

```
if (status !== PENDING)  
    throw new ConflictException(      ==);
```

/= agora — a entidade encapsula a máquina

```
const proposal = new TradeProposalEntity(status);  
proposal.accept();      /= estado válido → ACCEPTED  
                        /= estado terminal → lança sozinho
```

O service não tem mais nenhum `if` de status — só cria a entidade e chama o método.



PADRÕES ESTRUTURAIS · SRC/TRADES + PROVIDERS

Camadas limpas no módulo Trades

O módulo de trocas separa responsabilidades em camadas e fala com o resto do sistema por eventos — nunca por dependência direta.

● Repository

Isola o acesso ao banco (Prisma). O service não conhece o ORM — dá pra trocar de banco ou mockar em teste.
[trades.repository.ts](#)

● Service Layer

Concentra a regra de negócio (existe? status permite cancelar?) e mantém o controller fino.
[trades.service.ts](#)

● DTO

Contrato de entrada validado e documentado (Swagger), separando payload externo da entidade interna.
[create-trade.dto.ts](#)

● Injeção de Dependência

Cada camada recebe dependências pelo construtor (IoC do NestJS) — baixo acoplamento e fácil de testar.
[todos os construtores](#)

● Module

Encapsula o módulo e declara explicitamente controller, service e repositório.
[trades.module.ts](#)

● Observer (GoF)

Event bus por interfaces: o módulo dispara eventos sem depender de quem reage a eles.
[providers/events/](#)

4 — DEMONSTRAÇÃO

Vamos ao código rodando

Saindo dos slides — API NestJS no ar, documentação Swagger em `localhost:3000/docs`.

Criar proposta



Aceitar · State valida



Evento · Observer dispara



Wishlist · Strategy reage

Obrigado! 🙌